HW8 Interview Script — English Only

Simple language · Conversational · Short sentences

PART 1 — Project Overview (1-2 min)

> **[Show: project directory tree]**
>
> ```
> hw8/
> ├── mysql-service/main.go
> ├── dynamodb-service/main.go
> ├── terraform/main.tf
> └── test/run_test.py
> ```

So, hw8 is a database comparison project. The goal was simple: build the same shopping cart API twice. Once with MySQL. Once with DynamoDB. Then compare them.

Both services are written in Go. They expose the exact same three endpoints. I deployed everything on AWS using Terraform. Each service runs as an ECS Fargate container, behind its own load balancer.

The MySQL service connects to an RDS instance in a private subnet. The DynamoDB service uses the AWS SDK to talk to a DynamoDB table directly.

After both services were running, I ran identical load tests — 150 operations each — and compared performance, resource usage, and trade-offs.

> **[Show this diagram while speaking]**
>
> ```
> Internet
>  ├── ALB → ECS (mysql-service)    → RDS MySQL  (private subnet)
>  └── ALB → ECS (dynamodb-service) → DynamoDB   (managed by AWS)
>
> Both expose:
>  POST /shopping-carts
>  GET  /shopping-carts/:id
>  POST /shopping-carts/:id/items
> ```

PART 2 — Technical Implementation (5-6 min)

MySQL: Schema Design

> **[Show: `mysql-service/main.go`, lines 54–81]**

```
func createTables() {
    db.Exec(`
        CREATE TABLE IF NOT EXISTS carts (
            id          INT AUTO_INCREMENT PRIMARY KEY,
            customer_id INT NOT NULL,
            created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
            INDEX idx_customer_id (customer_id)   -- line 62
        ) ENGINE=InnoDB
    `)
    db.Exec(`
        CREATE TABLE IF NOT EXISTS cart_items (
            id         INT AUTO_INCREMENT PRIMARY KEY,
            cart_id    INT NOT NULL,
            product_id INT NOT NULL,
            quantity   INT NOT NULL,
            FOREIGN KEY (cart_id) REFERENCES carts(id) ON DELETE CASCADE,
-- line 73
            INDEX idx_cart_id (cart_id),
-- line 74
            UNIQUE KEY uq_cart_product (cart_id, product_id)
-- line 75
        ) ENGINE=InnoDB
    `)
}
```

I have two tables. `carts` stores the cart and the customer ID. `cart_items` stores each item inside a cart.

Three decisions I want to highlight.

First, `customer_id` has an index (line 62). That's for future queries like "show all carts for this customer." It's not needed for this assignment's test, but it's a good design choice.

Second, `ON DELETE CASCADE` on the foreign key (line 73). If a cart gets deleted, all its items are automatically deleted too. No orphaned rows.

Third, the `UNIQUE KEY` on `(cart_id, product_id)` (line 75). This is the key that makes the upsert work — I'll show that next.

MySQL: The Upsert

> **[Show: `mysql-service/main.go`, lines 210–214]**

```
_, err = db.Exec(`
    INSERT INTO cart_items (cart_id, product_id, quantity)
    VALUES (?, ?, ?)
    ON DUPLICATE KEY UPDATE quantity = quantity + VALUES(quantity)
`, id, body.ProductID, body.Quantity)
```

This is one SQL statement. It does two things at once. If the product is already in the cart, it adds to the quantity. If not, it inserts a new row. This is called an upsert.

The important thing is that it's atomic. There's no "check first, then write." That kind of read-then-write pattern creates race conditions when multiple requests come in at the same time. `ON DUPLICATE KEY UPDATE` avoids that entirely.

MySQL: Connection Pool

> **[Show: `mysql-service/main.go`, lines 42–43]**

```
db.SetMaxOpenConns(25)   // line 42
db.SetMaxIdleConns(25)   // line 43
```

MySQL needs a connection pool. Every TCP connection to the database takes time to open. Without pooling, every request pays that cost.

I set max 25 connections. That matches the assignment's requirement of supporting 100 concurrent sessions across multiple service instances.

In practice, during my 150-operation test, CloudWatch showed only 2 connections at peak. That's because my test was sequential, not concurrent.

DynamoDB: Table Design

> **[Show: `dynamodb-service/main.go`, lines 25–32]**

```
// Partition key: cart_id (String). No sort key needed.  -- line 26
type Cart struct {
    CartID     string     `dynamodbav:"cart_id"`      // line 28 —
partition key
    CustomerID int        `dynamodbav:"customer_id"`
    Items      []CartItem `dynamodbav:"items"`         // line 30 —
embedded list
    CreatedAt  string     `dynamodbav:"created_at"`
}
```

> **[Also show: `terraform/modules/dynamodb/main.tf`, lines 9–10]**

```
billing_mode = "PAY_PER_REQUEST"   // line 9
hash_key     = "cart_id"           // line 10
```

For DynamoDB, I used a single-table design. All cart data lives in one table.

The partition key is `cart_id`. I generate it as a nanosecond timestamp string:

> **[Show: `dynamodb-service/main.go`, line 77]**

```
cartID := strconv.FormatInt(time.Now().UnixNano(), 10)   // line 77
```

Nanosecond timestamps are essentially random. That means they distribute evenly across DynamoDB partitions. No hot partitions.

All cart items are embedded inside the same DynamoDB item as a list (line 30 in the struct). So one `GetItem` call returns the whole cart — no joins needed.

I didn't use a sort key. All three access patterns only ever need to look up by `cart_id`. A sort key would add complexity without any benefit.

DynamoDB: UpdateItem

> **[Show: `dynamodb-service/main.go`, lines 174–196]**

```
ddbClient.UpdateItem(context.TODO(), &dynamodb.UpdateItemInput{
    TableName: aws.String(tableName),
    Key: map[string]types.AttributeValue{
        "cart_id": &types.AttributeValueMemberS{Value: cartID},   // line
177-178
    },
    UpdateExpression: aws.String(
        "SET #items = list_append(if_not_exists(#items, :empty),
:new_item)"),  // line 179-180
    ConditionExpression: aws.String("attribute_exists(cart_id)"),
// line 181
    ...
})
```

This is the DynamoDB version of the MySQL upsert.

`list_append` (line 180) atomically adds the new item to the items list. `if_not_exists` handles the case where the list doesn't exist yet.

The `ConditionExpression` on line 181 is important. It checks that the cart actually exists before updating. If the cart doesn't exist, DynamoDB throws a `ConditionalCheckFailedException`.

> **[Show: `dynamodb-service/main.go`, lines 200–203]**

```
var condErr *types.ConditionalCheckFailedException   // line 200
if isConditionalCheckFailed(err, condErr) {          // line 201
    c.JSON(http.StatusNotFound, ...)                  // line 202
```

I catch that exception and return a 404. Without this, the service would silently create a broken item in DynamoDB. That's a correctness bug.

Infrastructure: Terraform

> **[Show: `terraform/main.tf`, lines 1–5 comments + module list]**

```
# Architecture:
#   Internet → ALB-MySQL    → ECS MySQL service  → RDS MySQL (private)
#   Internet → ALB-DynamoDB → ECS DynamoDB service → DynamoDB (managed)
```

> **[Show: `terraform/modules/rds/main.tf`, lines 14–30]**

```
resource "aws_db_instance" "this" {
    instance_class      = "db.t3.micro"        // line 17 — free tier
    publicly_accessible = false                // line 29 — private only
    skip_final_snapshot = true                 // line 30 — no cost on
destroy
```

All infrastructure is in Terraform. Six modules: network, logging, RDS, DynamoDB, two ALBs, two ECS services.

The RDS instance is `db.t3.micro`, MySQL 8.0, in a private subnet. Only the ECS tasks can reach it. `publicly_accessible = false` (line 29) enforces that.

The ECS task definition allocates 256 CPU units and 512MB memory:

> **[Show: `terraform/modules/ecs/main.tf`, lines 9–10]**

```
cpu    = "256"    // line 9
memory = "512"    // line 10
```

Config is passed through environment variables. The MySQL service gets `DB_DSN`. The DynamoDB service gets `TABLE_NAME` and `AWS_REGION` (lines 95–100 and 117–124 in `terraform/main.tf`).

PART 3 — Performance Data (5-6 min)

Overall Numbers

> **[Show: `report/report.md`, Overall Comparison Table]**

```
| Metric                  | MySQL    | DynamoDB | Winner    | Margin    |
|-------------------------|----------|----------|-----------|-----------|
```

```
| Avg Response Time (ms) | 110.95  | 100.74   | DynamoDB | 10.21 ms |
| P50 Response Time (ms) | 94.99   | 85.77    | DynamoDB | 9.22 ms  |
| P95 Response Time (ms) | 187.86  | 171.24   | DynamoDB | 16.62 ms |
| P99 Response Time (ms) | 223.09  | 185.45   | DynamoDB | 37.64 ms |
| Success Rate (%)       | 100.0   | 100.0    | Tie      | —        |
```

Both services got 100% success rate. So this is purely a latency comparison.

DynamoDB wins overall. It's 10ms faster on average.

But the more interesting number is P99. DynamoDB is 37ms faster at the tail. Why does that matter? In production, slow requests at P99 block server threads. At high concurrency, a 37ms tail latency difference compounds a lot.

Per-Operation Breakdown

> **[Show: `report/report.md`, Operation-Specific Breakdown]**

```
| Operation    | MySQL Avg (ms) | DynamoDB Avg (ms) | Faster By        |
|-------------|---------------|-------------------|-----------------|
| CREATE_CART | 102.35        | 103.69            | MySQL  +1.3 ms   |
| ADD_ITEMS   | 122.61        | 102.88            | DynamoDB +19.7 ms|
| GET_CART    | 107.89        | 95.65             | DynamoDB +12.2 ms|
```

MySQL actually wins on `create_cart`. But only by 1.3ms. That's basically noise.

The big gap is `add_items`. DynamoDB is 19.7ms faster. MySQL's upsert needs a unique constraint check plus a write with potential row locking. DynamoDB's `UpdateItem` is one atomic operation against a key-value store. Much simpler path.

`get_cart` shows DynamoDB 12ms faster. That makes sense too. MySQL runs two queries: one for the cart row, one for all the items rows. DynamoDB returns the whole cart in a single `GetItem`. One round trip vs two.

The Test Script

> **[Show: `test/run_test.py`, lines 27–52]**

```
for i in range(50):
    customer_id = random.randint(1, 10000)
    start = time.perf_counter()                          # line 29 — start
timer
    resp = requests.post(
        f"{base_url}/shopping-carts",
        json={"customer_id": customer_id}, timeout=10,
    )
    elapsed_ms = (time.perf_counter() - start) * 1000   # line 31 — stop
```

```
timer
    results.append({
        "operation": "create_cart",
        "response_time": round(elapsed_ms, 2),          # line 44
        "success": resp.status_code == 201,
        ...
    })
```

The test has three phases. 50 creates, 50 add-items, 50 gets. Same order for both services.

I use `time.perf_counter()` (lines 29 and 31) for timing. That's sub-millisecond precision. More accurate than `time.time()`.

The cart IDs from Phase 1 are passed into Phase 2 and 3 (line 63: `cart_id = cart_ids[i % len(cart_ids)]`). So all operations work on real persisted data.

The output format is identical for both services. That's what makes merging them into `combined_results.json` straightforward.

CloudWatch: MySQL CPU

> **[Show: MySQL CPU CloudWatch screenshot — `report/screenshots/mysql/mysql_cpu_utilization.png`]**

This one surprised me. MySQL CPU spiked to **58.4%** on just 150 sequential operations against a `db.t3.micro`.

That's a lot for a micro instance. And these were sequential operations, not concurrent. If I ran 100 concurrent users instead, I'd likely saturate the CPU immediately.

Write IOPS peaked at 251/s. Read IOPS was only 20/s. That confirms shopping carts are a write-heavy workload — which is expected.

CloudWatch: DynamoDB Capacity

> **[Show: DynamoDB CloudWatch screenshot — `report/screenshots/dynamodb/dy_read/dydb_write_capacity_units.png`]**

DynamoDB tells a completely different story. ConsumedWriteCapacityUnits peaked at **0.98**. ConsumedRead at **0.49**.

The moment the test finished, consumed capacity dropped to exactly zero. With PAY_PER_REQUEST billing, that means zero cost at idle. No instance running 24/7.

CloudWatch: ECS Memory

> **[Show: ECS memory CloudWatch screenshot — `report/screenshots/memoryutilization.png`]**

Here's the counter-intuitive finding. The DynamoDB service used **4x more memory** than MySQL. 1.37% vs 0.34%.

You'd expect the NoSQL service to be lighter. But the AWS SDK adds real overhead. SigV4 request signing for every API call. HTTPS connection management. JSON attribute marshaling and unmarshaling. A plain MySQL TCP connection is much lighter at the application layer.

So DynamoDB wins at the database layer. But it costs more at the application layer.

PART 4 — Design Decisions & Trade-offs (4-5 min)

The Core Decision

The key question is: what's your access pattern?

If every query is "give me the data for cart ID 42" — you only look up by one key — DynamoDB wins. It's faster, cheaper at idle, scales automatically, and you don't need to manage a connection pool.

If you need to ask "give me all carts abandoned today" or "show all orders for this customer" — you need SQL. DynamoDB can't answer those questions without a full table scan. That's expensive and slow. MySQL handles them naturally.

Four Scenarios

> **[Show: `report/report.md`, Scenario A–D section]**

Scenario A — Startup MVP (100 users/day): DynamoDB. The RDS instance costs ~$15/month even at idle. DynamoDB PAY_PER_REQUEST would cost cents per day at this scale. Zero idle cost is the deciding factor here.

Scenario B — Growing Business (10K users/day): MySQL. At this scale you start needing analytics — abandoned cart reports, customer purchase history. Those need SQL joins. DynamoDB can't do them without expensive scans.

Scenario C — High-Traffic Event (1M spike users): DynamoDB. MySQL CPU hit 58.4% on just 150 sequential ops. A 1M user spike would need heavy vertical scaling, probably with downtime. DynamoDB auto-scales with no intervention.

Scenario D — Global Platform (multi-region): DynamoDB. DynamoDB Global Tables handle multi-region replication natively. MySQL cross-region requires complex read replica setup and manual failover logic.

Polyglot Strategy

> **[Show: `report/report.md`, Polyglot Strategy table]**

```
| Component        | Database   | Why
|
```

```
|----------------|----------|-----------------------------------------
---|
| Shopping carts  | DynamoDB  | Key-value access, high write, TTL expiry
|
| User sessions   | DynamoDB  | Key-value, fast lookup, TTL built-in
|
| Product catalog | MySQL     | Search by category/price, complex joins
|
| Order history   | MySQL     | Relational data, audit trail, reporting
|
```

My conclusion is that neither database is universally better. The right answer depends on the access pattern.

For a complete e-commerce system, I'd use both. DynamoDB for carts and sessions — simple key-value, high write volume, natural expiry. MySQL for product catalog and orders — they need rich queries and audit trails.

PART 5 — What Went Wrong (2-3 min)

Bug 1 — Go Version Mismatch

> **[Show: `mysql-service/go.mod` first line, or Dockerfile]**

My first build failed. The Dockerfile was using `golang:1.22` as the base image. But when I ran `go mod tidy`, it upgraded `go.mod` to Go 1.23. That's because `gin-contrib/sse@v1.1.0` requires Go 1.23 minimum.

Trying to downgrade gin didn't work cleanly. The fix was simple once I found the cause: update the Dockerfile base image to `golang:1.23`. They have to match exactly.

Lesson: always make sure your Dockerfile Go version matches what's in `go.mod`.

Bug 2 — ECR Timing with Terraform

Terraform created the ECS services before I had pushed the Docker images to ECR. So the ECS tasks immediately failed — they tried to pull an image that didn't exist yet.

The immediate fix was:

```
aws ecs update-service --cluster <cluster> \
  --service <service> --force-new-deployment
```

But that's a workaround, not a real fix. The cleaner solution is to separate the Terraform apply into two steps. Create ECR first. Push the images. Then create the ECS services. This is a classic chicken-and-egg problem in infrastructure-as-code deployments.

Bug 3 — `depends_on` with a Variable

> **[Show: `terraform/modules/ecs/main.tf`, lines 49–53]**

```
load_balancer {
    target_group_arn = var.target_group_arn   // line 50 — implicit
dependency
    container_name   = ...
    container_port   = ...
}
```

In the ECS module, I originally wrote `depends_on = [var.alb_listener_arn]`. Terraform rejected it. `depends_on` only accepts resource references. Not variable values.

The fix was to delete the `depends_on` entirely. The ECS service already has an implicit dependency on the ALB through `target_group_arn` on line 50. Terraform reads the attribute graph and infers the dependency automatically. The explicit `depends_on` was redundant and wrong.

Final Reflection

Before this assignment, I thought "DynamoDB is faster" was just marketing. After actually measuring it — the advantage is real, but it's more nuanced than I expected.

DynamoDB wins on P99 latency and write-heavy operations. But it uses 4x more application memory because of SDK overhead. The 10ms average difference isn't even the most important reason to choose one over the other.

The real decision comes down to access patterns and operational complexity. If your query is "get by ID" — use DynamoDB. If your query is "give me everything matching these conditions" — use MySQL.

Total: ~18–20 minutes